be another native application storing data on a disk. The disjoint-set client will run on the same intranet, possibly even on the same machine, as the e-commerce server; it will have a cron job to keep the persistent storage in sync with the in-memory version (this could happen every few seconds or asynchronously after each operation). Moreover, it will respond to calls from the e-commerce site, querying the in-memory storage or, when needed, calling the web server (and, in case not all the data will fit in memory, it will also take care of swaps).

So, let's assume we agree on the need to write a class that takes care of the whole problem, keeping track of which (disjoint) subset an element belongs to, and encapsulating all the logic in it. We are going to delay the discussion on implementation for now: before focusing on the details, let's discuss its public API and behavior.

Depending on the size of the catalog, we could even fit such a data structure in memory, but let's instead assume that we set up a REST service (illustrate in figure 5.3) based on a persisted Memcached-like storage[1], something like *Redis*. Durability of the data is important in this case, because obviously, in our example, the monitoring activity over the items will last for years, and we don't want to recompute the whole disjoint-set structure every time there is a change or a new product is added.

Alternatively, if the size of the universe[2] is small enough to fit in memory, it could be possible to imagine a